Indian Institute of Technology, Gandhinagar

CS - 327 Compilers
Assignment 2 - Lexical Analyser
Lexical Analyzer

Prof. Shouvick Mondal

Team:
Sailaja Mudu (23110288)
Harsha Vardhan Nayak (23110349)
Rushitha Thipparapu (23110338)

Design of the Lexical Analyzer

Introduction

A compiler translates high-level programming language code into machine code. The compilation process is divided into several phases, and the first phase is Lexical Analysis. Lexical analysis reads the source program as a stream of characters and groups them into meaningful sequences called tokens. Each token represents a logical unit such as:

- Keywords
- Identifiers
- Constants
- Operators
- Punctuators

In this assignment, a lexical analyzer for a simplified C-like programming language called MiniC was implemented using FLEX (Fast Lexical Analyzer Generator). The analyzer follows the provided MiniC language specification and supports token detection, error reporting, symbol table generation, and summary statistics.

Token Recognition Strategy

The lexical analyzer was implemented using FLEX, which generates a scanner based on regular expressions. The analyzer reads the input source code character by character and groups sequences of characters into meaningful tokens according to predefined patterns.
Each token category is defined using a regular expression. When FLEX finds a match, it executes the associated action code to:

1. Classify the token type
2. Print formatted token details
3. Update counters
4. Maintain symbol table (for identifiers)

Token Classification Process

The analyzer identifies the following token categories:

Token Category	Recognition Method
Keywords	Matched as identifiers and verified using keyword table int, long, short, float, double, char, void,if, else, while, do, for, switch, case, default break, continue, return, struct, sizeof

Integer Literals	[0-9]+
Hex Literals	0[xX][0-9a-fA-F]+
Identifiers	[a-zA-Z_][a-zA-Z0-9_]*
Floating Literals	{DIGIT}+\.{DIGIT}+([eE][+-]?{DIGIT}+)?
Character Literals	\'(\\.|[^\\\']\)\'
String Literals	\"([^\\\"\n]|\\.)*\"
Operators	<=>"|"<="|">="|"=="|"!="|"++"|"--"|"&&"|"||" { [+\-*/%=<>!&|^~.]
Punctuators	[(){}\[\];,:]
Comments	// and /* */ patterns

Keyword Recognition Strategy

Keywords are not given separate regular expressions. Instead:

1. The identifier pattern {ID} matches the lexeme.
2. A function is_keyword() checks whether the lexeme exists in the keyword list.
3. If found → token classified as KEYWORD.
4. Otherwise → classified as IDENTIFIER.

This avoids writing separate rules for each keyword and improves maintainability.

Symbol Table Handling

When a valid identifier is detected:

- It is inserted into a symbol table.
- Only the first occurrence is stored.
- The line number of first appearance is recorded.

This enables tracking of user-defined entities.

Output Format Strategy

Each recognized token is printed with:

- Serial Number
- Token Type
- Lexeme
- Row Number
- Column Number
- Length of lexeme

This structured output improves readability and debugging.

Rule Ordering Strategy

Rule ordering is a critical aspect of FLEX-based lexical analysis.
FLEX follows two principles:

1. Longest Match Rule – The longest possible match is selected.
2. First Rule Priority – If multiple patterns match the same length, the rule appearing first in the file is selected.

Therefore, correct ordering of rules is essential to avoid incorrect tokenization.

1. Multi-Character Operators Before Single-Character Operators

Example:
"<=>"|"<="|">="|"=="|"!="|"++"|"--"|"&&"|"||"
[+\-*/%=<>!&|^~.]

If the single-character rule appeared first, || would be tokenized as two | operators instead of one ||.
Thus:
✔ Multi-character operators must appear before single-character operators.

2. Floating-Point Before Integer Literals

Example:
Input: 3.14

If the integer rule [0-9]+ appeared before the float rule, FLEX would tokenize:
3 → INTEGER
. → OPERATOR
14 → INTEGER

To prevent this, the FLOAT rule is placed before INT.

3. Hexadecimal Before Invalid Identifier Rule

Example: 0x1A3F

If the invalid identifier rule [0-9]+[a-zA-Z_][a-zA-Z0-9_]* appeared first, FLEX might incorrectly split the token.
Therefore:
✔ HEX rule must appear before INVALID_IDENTIFIER rule.

4. Unterminated String Before Valid String Rule

Example: "Hello

If only the valid string rule is present, the error may not be detected properly.
Thus:
✔ Unterminated string rule must be placed before valid string rule.

5. Most Specific Rules Before General Rules

General rule ordering principle followed:

1. Error-specific rules
2. Complex literals (FLOAT, HEX)
3. Invalid patterns
4. Simple literals (INT)
5. Identifiers
6. Operators (multi-char → single-char)
7. Punctuators
8. Whitespace
9. Catch-all rule (.)

This ensures correct and predictable token recognition.

Error Handling Mechanisms

The lexical analyzer detects and reports invalid patterns to ensure strict compliance with the MiniC specification.

1. **Invalid Identifier**

   Identifiers must begin with a letter or underscore.
   Tokens starting with digits followed by letters are invalid.
   Example:
   123abc

   Detection Rule:
   [0-9]+[a-zA-Z_][a-zA-Z0-9_]*

   Such tokens are classified as INVALID_IDENTIFIER and counted as lexical errors.

2. **Unterminated String Literal**

   A string must start and end with double quotes.
   If a newline appears before the closing quote, it is treated as an error.
   Example:
   "Hello World

   Detection Rule:
   \"([^\\\"\n]|\\.)*\n

   The token is reported as UNTERMINATED_STRING, and line tracking is updated.

3. **Unclosed Multi-line Comment**

   Multi-line comments must end with */.
   If the file ends while still inside comment state, an error is reported.
   Detection Rule:

```
<<EOF>> {
  if (YY_START == COMMENT)
      printf("Unclosed comment");
}
```

   This ensures incomplete comments are properly detected.

4. **Invalid Characters**

   Any character not matching valid token patterns is classified as invalid.
   Catch-all Rule:
   .

Such characters are reported as INVALID_TOKEN.

The analyzer ensures robust error detection by:

- Using specific regular expressions for malformed tokens
- Checking comment state at EOF
- Placing error rules before general rules
- Maintaining error counters for reporting

This guarantees reliable and specification-compliant lexical analysis.